III. EDA-Data Cleaning

AIM:

- Handling missing values: detection, filling, and dropping
- Removing duplicates and unnecessary data
- Data type conversion and ensuring consistency
- Normalize data (e.g., standardization, min-max scaling).

PROCEDURE:

1. Import the required Python libraries, such as Pandas and NumPy, for data preprocessing and manipulation.
2. Load the dataset into a Pandas DataFrame using pd.read_csv().
3. Detect missing values in the dataset using isnull().sum().
4. Handle missing values by filling them with suitable values using fillna() or removing rows and columns using dropna().
5. Identify and remove duplicate records using duplicated() and drop_duplicates().
6. Remove unnecessary columns or irrelevant data that are not required for analysis.
7. Convert columns to appropriate data types using astype() and ensure consistency in the dataset.
8. Normalize numerical data using standardization (StandardScaler) or Min-Max scaling (MinMaxScaler).
9. Display the cleaned and normalized dataset and verify that missing values, duplicates, and data inconsistencies have been handled successfully.

PROGRAM AND OUTPUT:

```
# Importing required libraries

import pandas as pd

import numpy as np

from google.colab import files
```

```
# Upload dataset

uploaded = files.upload()

# Load dataset

df=pd.read_csv("student_missing_data.csv")


# 1. Handling Missing Values


# Check missing values
print(df.isnull().sum())

# Fill numerical columns with mean
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())

# Fill categorical columns with mode
cat_cols = df.select_dtypes(include='object').columns
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

# Verify
print("\nMissing values after handling:")
print(df.isnull().sum())

# Save cleaned dataset
df.to_csv("cleaned_dataset.csv", index=False)

# Download cleaned dataset
files.download("cleaned_dataset.csv")
```

```
StudentID           0
Name                0
Gender              0
Age                 4
StudyHours          0
AttendancePercent   4
ExamScore           4
dtype: int64

Missing values after handling:
StudentID           0
Name                0
Gender              0
Age                 0
StudyHours          0
AttendancePercent   0
ExamScore           0
dtype: int64
```

```
# 2. Removing Duplicates
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
print(f"\nRemoved {initial_rows - df.shape[0]} duplicate rows.")
```

```
Removed 0 duplicate rows.
```

```
# 3. Data Type Conversion
# Convert 'Dependents' to numeric (replace '3+' with 3)
df['Dependents'] = df['Dependents'].replace('3+', 3).fillna(0).astype(int)
# 4. Ensuring Categorical Consistency
for col in ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']:
    df[col] = df[col].str.strip().str.capitalize()
# 5. Normalization
min_max_scaler = MinMaxScaler()
scale_cols = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']
```

```
df[scale_cols] = min_max_scaler.fit_transform(df[scale_cols])
scaler = StandardScaler()
df[['Credit_History']] = scaler.fit_transform(df[['Credit_History']])
# 6. Final Overview
print("\nCleaned Data Summary:")
print(df.info())
print(df.head())
```

	StudentID	Name	Gender	Age	StudyHours	AttendancePercent	ExamScore
0	S01	Aarav	M	20.000000	12	88.0	78.000000
1	S02	Diya	F	19.000000	8	82.0	65.000000
2	S03	Kabir	M	21.000000	15	75.0	90.000000
3	S04	Ananya	F	20.000000	6	80.0	73.761905
4	S05	Vivaan	M	22.000000	10	95.0	82.000000
5	S06	Ishita	F	19.000000	9	70.0	58.000000

```
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("cleaned_dataset.csv")

# Select numerical columns
numerical_cols = df.select_dtypes(include=np.number).columns

# Detect and remove outliers using IQR
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    print(f"{col}: {(df[(df[col] < lower) | (df[col] > upper)]).shape[0]} outliers")

    df = df[(df[col] >= lower) & (df[col] <= upper)]

# Save cleaned dataset
df.to_csv("dataset_without_outliers.csv", index=False)

print("Outlier analysis completed.")
```

```
print("Final Dataset Shape:", df.shape)
```

```
Age: 2 outliers
StudyHours: 0 outliers
AttendancePercent: 1 outliers
ExamScore: 0 outliers
Outlier analysis completed.
Final Dataset Shape: (22, 7)
```

RESULT:

Thus, the program was written and executed successfully.